

Assignment 3: Neural Network Classification

Name: Yerraiahgari Ram Charan Goud

Roll No: 241212

Task 1: Data Preparation

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical
import numpy as np

iris = load_iris()
X = iris.data
y = iris.target
```

- **load_iris:** Imports the standard Iris dataset containing 150 samples with 4 features (sepal length, sepal width, petal length, petal width).
- **X and y:** `X` stores the feature vectors, and `y` stores the target class labels (0 for setosa, 1 for versicolor, 2 for virginica).

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

- **train_test_split:** Divides the data into an 80% training set (120 samples) and a 20% testing set (30 samples). The `random\_state=42` ensures reproducibility.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

- **StandardScaler:** Normalizes the features to have a mean of zero and a variance of one, which helps the neural network converge faster.

- **to_categorical:** Applies one-hot encoding to the integer labels (e.g., class 0 becomes [1, 0, 0]). This makes the target variables suitable for multi-class categorical crossentropy loss.

Task 2: Neural Network Construction

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(8, input_shape=(4,), activation='relu'))
model.add(Dense(3, activation='softmax'))
```

- **Sequential:** Initializes a linear stack of network layers.
- **Hidden Layer:** A fully connected (Dense) layer with 8 neurons. It expects an input of 4 features. The **ReLU** (Rectified Linear Unit) activation introduces non-linearity, allowing the model to learn complex patterns.
- **Output Layer:** Contains 3 neurons corresponding to the 3 target classes. The **Softmax** activation converts the raw output scores into a normalized probability distribution where the sum of all probabilities equals 1.

Task 3: Model Compilation and Training

```
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=100, batch_size=5, verbose=0)
```

- **Adam Optimizer:** An adaptive learning rate optimization algorithm known for its efficiency in deep learning.
- **Categorical Crossentropy:** The standard loss function for multi-class classification tasks using one-hot encoded labels.
- **Training (fit):** The model iterates over the entire training dataset 100 times (`epochs=100`), updating weights after every 5 samples (`batch\_size=5`).

Task 4: Model Evaluation

```
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

The trained model is evaluated against the unseen test dataset to calculate the final loss and accuracy metrics.